

Robot Cat

Ekspansja: mapa domu i rozpoznawanie wizyjne

Punkt wyjścia: kamera (Camera Module 3), VL53L5CX i BNO085 są w planie zakupowym. Raspberry Pi 5 + Raspberry Pi AI HAT+ 13 TOPS (Hailo-8L) nie są jeszcze zdecydowane — plan zakupowy wciąż zakłada Pi 4B bez akceleratora (patrz plan-zakupowy.pdf). Cała ekspansja opisana tu wymaga tej zmiany, bo Pi 4B nie ma złącza PCIe na Hailo. Ten dokument rozpisuje, czego jeszcze brakuje, żeby dojść od „ma kamerę” do „idź do pokoju A, zobacz co jest na stole” — pod warunkiem, że Pi 5 + HAT zostaną kupione.

Wniosek w jednym zdaniu

Brakujące warstwy to mapowanie (SLAM wizyjny), nazwane miejsca na mapie, rozpoznawanie obiektów i prosty sekwencer łączący je w jedną komendę — żadna z nich nie wymaga nowego sprzętu ponad Pi 5 + Hailo-8L, którego zakup jest osobną, wciąż otwartą decyzją, i żadna nie działa w izolacji od reszty.

1. Cztery warstwy między „ma kamerę” a „rozumie dom”

Warstwa	Zadanie	Bez tego...
Mapowanie (SLAM)	zbudować mapę pomieszczeń z kamery + IMU	robot nie wie, gdzie jest ani jak wygląda dom
Nazwane miejsca	powiązać obszar mapy z etykietą „pokój A”	mapa jest, ale nie da się do niej odwołać po nazwie
Rozpoznawanie obiektów	wykryć i zaklasyfikować to, co widzi kamera	robot dojedzie, ale nie powie, co widzi na stole
Sekwencer zadań	połączyć „jedź” + „rozpoznaj” w jedną komendę	trzeba by to odpalać ręcznie, krok po kroku

2. Mapowanie: SLAM wizyjny

Bez LiDAR-u (decyzja podjęta wcześniej) mapowanie musi iść z kamery i IMU, które już mamy w planie. To dokładnie do czego służy RTAB-Map — pakiet ROS 2 do wizyjnego SLAM-u, z potwierdzonym wsparciem dla Jazzy. Buduje mapę 3D albo czystą siatkę zajętości 2D (do nawigacji) z jednej kamery RGB i danych IMU, wykorzystując wykrywanie zamknięć pętli (rozpoznawanie „już tu byłem”), żeby mapa się nie rozjeżdżała przy dłuższym chodzeniu.

Przykład: rtabmap_ros odbiera obraz z kamery i dane z BNO085, i publikuje gotową siatkę zajętości na topicu, którego Nav2 używa bezpośrednio do planowania trasy — nie trzeba pisać własnego mapowania od zera.

3. Nazwane miejsca i dojście do nich

Gdy mapa istnieje, Nav2 (standardowy stos nawigacji ROS 2) umie dojechać do zadanej pozycji (x, y) na tej mapie. „Pokój A” to w praktyce współrzędne zapisane raz — najprościej: przejechać robotem po domu ręcznie jeden raz, zaznaczyć środek każdego pokoju jako punkt nazwany, i zapisać. Komenda „idź do pokoju A” to wtedy jedno wywołanie Nav2 z zapisanymi współrzędnymi, nie nowy problem nawigacyjny.

4. Rozpoznawanie obiektów — z jednym uczciwym zastrzeżeniem

Standardowy YOLOv8, skompilowany pod Hailo-8L (ten sam model, którego FPS mierzyliśmy wcześniej — ~137 FPS dla wariantu n) rozpoznaje 80 klas ze zbioru COCO, co obejmuje większość typowych przedmiotów na stole: kubek, telefon, laptop, książkę, pilot, klawiaturę, butelkę. To jest gotowe do użycia dziś, bez dodatkowego treningu.

Czego jeszcze nie da się zrobić

Prawdziwie otwarte rozpoznawanie „zobacz cokolwiek, po nazwie, bez wcześniej ustalonej listy” (modele typu YOLO-World) nie jest jeszcze wspierane na Hailo-8/8L — Ultralytics wprost odrzuca tę rodzinę modeli przy eksporcie pod ten akcelerator. Na start trzeba więc zamknąć się w zestawie klas COCO, nie w swobodnym „co to jest” dla dowolnego przedmiotu. To się może zmienić z kolejną wersją oprogramowania Hailo — wart sprawdzenia przy realizacji, nie zakładania na dziś.

Przykład: węzeł ROS 2 odbiera klatkę z kamery, przepuszcza ją przez YOLOv8 na Hailo-8L, i publikuje listę wykrytych obiektów z pozycją w kadrze — dokładnie to demonstruje [hailo-rpi5-examples](#), oficjalny zestaw przykładów Hailo na Raspberry Pi 5.

5. Sekwencer: „idź do pokoju A, zobacz co jest na stole”

Ostatni element to nie nowy model, tylko kolejność wywołań: (1) Nav2 dojeżdża do zapisanej pozycji pokoju A, (2) po dojechaniu robot obraca głowę w stronę stołu (mikroserwo już w planie), (3) węzeł YOLOv8 rozpoznaje klatkę i zwraca listę obiektów. To da się złożyć jako prosty skrypt Python wywołujący akcje ROS 2 po kolei — nie wymaga własnego frameworku do planowania zadań na tym etapie.

6. Koszt energetyczny tej ekspansji

Hailo-8L dolicza 1-1,5 W aktywnego liczenia do budżetu, który już liczy ~50-70 W z samych serw — zaokrąglenie błędu pomiaru, nie realna zmiana czasu pracy na baterii. Tabela czasu pracy z dokumentu o napędach (3000 mAh → ~30-36 min, 5000 mAh → ~50-60 min) obowiązuje bez zmian z tą ekspansją.

Co ten dokument świadomie pomija

Konkretne parametry RTAB-Map (rozmiar mapy, częstotliwość zamknięć pętli), sposób etykietowania pokoi w interfejsie użytkownika, i finetuning YOLOv8 pod niestandardowe klasy (np. konkretne przedmioty domowe poza zbiorem COCO) — to decyzje do podjęcia przy implementacji, nie teraz. Żadna z czterech warstw nie jest jeszcze wdrożona; to plan, nie stan.